new Date()

📄 Título

new Date() – El constructor de fechas en JavaScript

📝 Definición

`new Date()` es el **constructor** de JavaScript que crea una nueva instancia de objeto `Date` que representa un **momento específico en el tiempo**.

Puede ser llamado de diferentes maneras para crear fechas a partir de timestamps, strings, componentes de fecha/hora, o la fecha actual.

🧠 ¿Qué es?

Es el mecanismo principal para trabajar con fechas y horas en JavaScript.

Crea un objeto `Date` que encapsula un timestamp interno (milisegundos desde el 1 de enero de 1970 UTC) y proporciona métodos para manipular y formatear esa fecha.

Características clave:

- Representa un **instante exacto en el tiempo**.
- Almacena internamente un **timestamp en milisegundos** (UTC).
- Proporciona métodos para obtener/establecer componentes (año, mes, día, hora, etc.).
- Soporta **zonas horarias** (métodos locales y UTC).
- Es **mutable** (los métodos set modifican el objeto).

🎯 ¿Para qué sirve?

- Crear fechas y horas en JavaScript.
- Representar **momentos específicos** en el tiempo.
- Realizar **cálculos de fechas** (sumar/restar días, horas, etc.).
- Comparar **fechas** y **ordenar** cronológicamente.

- Formatear fechas para **visualización**.
- Extraer **componentes** (año, mes, día, hora, etc.).
- Implementar **calendarios, agendas y cronómetros**.
- Generar **timestamps** para logs, IDs y métricas.

⚙️ ¿Cómo funciona?

El constructor `Date` tiene **cuatro formas principales** de ser llamado:

1. `new Date()`: Fecha y hora actuales (timestamp actual).
2. `new Date(timestamp)`: Desde un timestamp en milisegundos.
3. `new Date(dateString)`: Desde una string de fecha.
4. `new Date(year, month, day, hours, minutes, seconds, ms)`: Desde componentes individuales (hora local).

Comportamiento:

- Cuando se usa `Date()` sin `new`, devuelve un **string**, no un objeto.
- Los meses son **base 0** (0 = enero, 11 = diciembre).
- Los años se especifican con **4 dígitos** (ej: 2024).
- La fecha se interpreta en la **zona horaria local** del sistema.
- Si los parámetros están fuera de rango, se **ajustan automáticamente** (overflow).

📝 Sintaxis / Estructura (JavaScript)

```
// 1. Fecha actual
new Date()

// 2. Desde timestamp (milisegundos desde Epoch)
new Date(timestamp)

// 3. Desde string de fecha
new Date(dateString)
```

```
// 4. Desde componentes individuales (hora local)
new Date(year, month)
new Date(year, month, day)
new Date(year, month, day, hours)
new Date(year, month, day, hours, minutes)
new Date(year, month, day, hours, minutes, seconds)
new Date(year, month, day, hours, minutes, seconds, millise
conds)

// 5. Sin new (devuelve string)
Date() // "Mon Dec 25 2024 15:30:45 GMT+0100 (CET)"
```

💡 Ejemplos

```
// 1. Fecha y hora actual
const now = new Date();
console.log(now); // 2024-12-25T15:30:45.123Z

// 2. Desde timestamp (milisegundos)
const timestamp = 1735084800000; // 2024-12-25 00:00:00 UTC
const dateFromTimestamp = new Date(timestamp);
console.log(dateFromTimestamp.toISOString()); // "2024-12-2
5T00:00:00.000Z"

// 3. Desde string de fecha
const dateFromString = new Date('2024-12-25');
console.log(dateFromString.getFullYear()); // 2024
console.log(dateFromString.getMonth()); // 11 (diciembre)

// 4. Desde componentes individuales
const dateComponents = new Date(2024, 11, 25, 15, 30, 45, 5
00);
console.log(dateComponents.getFullYear()); // 2024
console.log(dateComponents.getMonth()); // 11 (diciembre)
console.log(dateComponents.getDate()); // 25
console.log(dateComponents.getHours()); // 15
```

```
console.log(dateComponents.getMinutes()); // 30
console.log(dateComponents.getSeconds()); // 45
console.log(dateComponents.getMilliseconds()); // 500

// 5. Solo año y mes
const dateYearMonth = new Date(2024, 11);
console.log(dateYearMonth.getDate()); // 1 (primer día del
mes)

// 6. Año, mes y día
const dateYMD = new Date(2024, 11, 25);
console.log(dateYMD.toISOString()); // "2024-12-25T00:00:0
0.000Z" (UTC)

// 7. Fecha actual sin new (string)
const dateString = Date();
console.log(typeof dateString); // "string"
console.log(dateString); // "Mon Dec 25 2024 15:30:45 GMT+0
100 (CET)"

// 8. Crear fecha a partir de timestamp actual
const nowTimestamp = Date.now();
const dateFromNow = new Date(nowTimestamp);
console.log(nowTimestamp === dateFromNow.getTime()); // tru
e

// 9. Fechas inválidas
const invalidDate1 = new Date('fecha inválida');
console.log(invalidDate1.toString()); // "Invalid Date"
console.log(isNaN(invalidDate1.getTime())); // true

const invalidDate2 = new Date(2024, 13, 32); // Mes 13, día
32
console.log(invalidDate2.getMonth()); // 2 (marzo) - ¡se aj
ustó automáticamente!

// 10. Fechas antes del Epoch
const oldDate = new Date(-31536000000); // 1969-01-01
```

```
console.log(oldDate.getFullYear()); // 1969

// 11. Diferentes formatos de string
const formats = [
  new Date('2024-12-25'),
  new Date('2024/12/25'),
  new Date('Dec 25 2024'),
  new Date('2024-12-25T15:30:45'),
  new Date('2024-12-25T15:30:45Z'),
  new Date('2024-12-25T15:30:45+02:00')
];

formats.forEach(d => console.log(d.toISOString()));

// 12. Ajuste automático de parámetros
const autoAdjust = new Date(2024, 0, 32); // 32 de enero
console.log(autoAdjust.getDate()); // 1 (febrero)
console.log(autoAdjust.getMonth()); // 1 (febrero)

// 13. Crear fecha para un día específico de la semana
function getDateForWeekday(year, month, weekday, weekNumber
= 1) {
  const date = new Date(year, month, 1);
  const firstDay = date.getDay();
  const diff = (weekday - firstDay + 7) % 7;
  date.setDate(1 + diff + (weekNumber - 1) * 7);
  return date;
}

console.log(getDateForWeekday(2024, 11, 1, 3)); // 3er lune
s de diciembre 2024

// 14. Copiar fecha
function cloneDate(date) {
  return new Date(date.getTime());
}

const original = new Date(2024, 11, 25);
```

```
const copy = cloneDate(original);
console.log(copy.getTime() === original.getTime()); // true

// 15. Crear fecha a medianoche
function createMidnight(year, month, day) {
  return new Date(year, month, day, 0, 0, 0, 0);
}

console.log(createMidnight(2024, 11, 25)); // 2024-12-25 0
0:00:00.000
```

⚠️ Errores comunes

- **Usar `Date()` sin `new` esperando un objeto:**

```
// ❌ MAL - devuelve string
const date = Date();
console.log(typeof date); // "string"
console.log(date.getTime()); // Error: date.getTime is n
ot a function

// ✅ BIEN
const date = new Date();
console.log(date.getTime()); // 1703539200000
```

- **Meses base 0:**

```
// ❌ MAL - diciembre es 11, no 12
const date = new Date(2024, 12, 25);
console.log(date.getMonth()); // 0 (enero del año siguie
nte)

// ✅ BIEN
const date = new Date(2024, 11, 25);
console.log(date.getMonth()); // 11 (diciembre)
```

- **Años de 2 dígitos**:

```
// ❌ MAL - 99 se interpreta como 99 d.C. (no 1999)
const date = new Date(99, 11, 25);
console.log(date.getFullYear()); // 99

// ✅ BIEN
const date = new Date(1999, 11, 25);
console.log(date.getFullYear()); // 1999
```

- **Formatos de string ambiguos**:

```
// ⚠️ Los strings pueden interpretarse diferente según e
l navegador
const date = new Date('01/02/2024');
// En US: 2 de enero, en UK: 1 de febrero

// ✅ Mejor usar formato ISO (YYYY-MM-DD)
const date = new Date('2024-01-02'); // 2 de enero
```

- **Zonas horarias**:

```
// ⚠️ Los strings ISO sin hora se interpretan en UTC
const date1 = new Date('2024-12-25'); // 2024-12-25 00:0
0:00 UTC
const date2 = new Date(2024, 11, 25); // 2024-12-25 00:0
0:00 local

console.log(date1.getHours()); // 0 (UTC)
console.log(date2.getHours()); // 0 (local)
```

💼 Casos de uso

1. **Sistema de registro de usuarios con fecha de creación**:

```
class User {
  constructor(name, email) {
    this.id = this.generateId();
    this.name = name;
    this.email = email;
    this.createdAt = new Date();
    this.updatedAt = new Date();
    this.lastLogin = null;
  }

  generateId() {
    return `${Date.now()}-${Math.random().toString(36).s
ubstr(2, 6)}`;
  }

  login() {
    this.lastLogin = new Date();
    return this.lastLogin;
  }

  updateProfile(data) {
    Object.assign(this, data);
    this.updatedAt = new Date();
    return this;
  }

  getAccountAge() {
    const now = new Date();
    const age = now.getTime() - this.createdAt.getTime
();
    const days = Math.floor(age / (1000 * 60 * 60 * 2
4));
    const hours = Math.floor((age % (1000 * 60 * 60 * 2
4)) / (1000 * 60 * 60));
    return { days, hours };
  }
```

```
}

const user = new User('Juan', 'juan@email.com');
console.log(user.createdAt); // Fecha de creación
user.login();
console.log(user.lastLogin); // Último login
```

2. **Generador de informes con períodos**:

```
class ReportGenerator {
  constructor() {
    this.reports = [];
  }

  generateReport(name, period = 'monthly') {
    const now = new Date();
    let startDate, endDate;

    switch(period) {
      case 'daily':
        startDate = new Date(now.getFullYear(), now.getM
onth(), now.getDate());
        endDate = new Date(startDate);
        endDate.setDate(endDate.getDate() + 1);
        break;
      case 'weekly':
        startDate = new Date(now);
        startDate.setDate(now.getDate() - now.getDay());
        startDate.setHours(0, 0, 0, 0);
        endDate = new Date(startDate);
        endDate.setDate(endDate.getDate() + 7);
        break;
      case 'monthly':
        startDate = new Date(now.getFullYear(), now.getM
onth(), 1);
        endDate = new Date(now.getFullYear(), now.getMon
th() + 1, 0);
```

```
        break;
      case 'yearly':
        startDate = new Date(now.getFullYear(), 0, 1);
        endDate = new Date(now.getFullYear(), 11, 31);
        break;
      default:
        startDate = new Date();
        endDate = new Date();
    }

    const report = {
      name,
      period,
      generatedAt: new Date(),
      startDate,
      endDate,
      data: this.getReportData(startDate, endDate)
    };

    this.reports.push(report);
    return report;
  }

  getReportData(startDate, endDate) {
    // Simular datos del reporte
    return {
      total: Math.floor(Math.random() * 1000),
      average: Math.floor(Math.random() * 100),
      dateRange: `${startDate.toISOString().split('T')
[0]} -${endDate.toISOString().split('T')[0]}`
    };
  }
}

const generator = new ReportGenerator();
const monthlyReport = generator.generateReport('Ventas',
'monthly');
console.log(monthlyReport);
```

3. **Sistema de citas y calendario**:

```
class CalendarManager {
  constructor() {
    this.appointments = [];
  }

  createAppointment(title, date, duration = 30) {
    const start = new Date(date);
    // Redondear a minutos exactos
    start.setMinutes(start.getMinutes(), 0, 0);

    const end = new Date(start);
    end.setMinutes(end.getMinutes() + duration);

    this.appointments.push({
      id: Date.now() + '-' + Math.random().toString(36).
substr(2, 4),
      title,
      start,
      end,
      duration,
      created: new Date()
    });

    return this.appointments[this.appointments.length -
1];
  }

  getAppointmentsByDay(date) {
    const target = new Date(date);
    target.setHours(0, 0, 0, 0);

    return this.appointments
      .filter(a => {
        const start = new Date(a.start);
        start.setHours(0, 0, 0, 0);
```

```
        return start.getTime() === target.getTime();
      })
      .sort((a, b) => a.start - b.start);
  }

  getUpcomingAppointments(days = 7) {
    const now = new Date();
    const future = new Date(now);
    future.setDate(future.getDate() + days);

    return this.appointments
      .filter(a => a.start >= now && a.start <= future)
      .sort((a, b) => a.start - b.start);
  }

  getAvailableSlots(date, duration = 30) {
    const appointments = this.getAppointmentsByDay(dat
e);
    const slots = [];
    const start = new Date(date);
    start.setHours(9, 0, 0, 0); // 9:00 AM
    const end = new Date(date);
    end.setHours(17, 0, 0, 0); // 5:00 PM

    let current = new Date(start);
    while (current < end) {
      const slotEnd = new Date(current);
      slotEnd.setMinutes(current.getMinutes() + duratio
n);

      const isBooked = appointments.some(a =>
        a.start.getTime() === current.getTime()
      );

      if (!isBooked) {
        slots.push({
          start: new Date(current),
          end: slotEnd,
```

```
          time: current.toLocaleTimeString('es-ES', {
            hour: '2-digit',
            minute: '2-digit'
          })
        });
      }

      current.setMinutes(current.getMinutes() + duratio
n);
    }

    return slots;
  }
}

const calendar = new CalendarManager();
calendar.createAppointment('Reunión', new Date(2024, 6,
15, 10, 0), 30);
console.log(calendar.getAvailableSlots(new Date(2024, 6,
15)));
```

4. **Sistema de vencimiento de licencias**:

```
class LicenseManager {
  constructor() {
    this.licenses = new Map();
  }

  createLicense(userId, durationDays = 30) {
    const now = new Date();
    const expiry = new Date(now);
    expiry.setDate(expiry.getDate() + durationDays);

    const license = {
      userId,
      issuedAt: new Date(now),
      expiresAt: expiry,
```

```
      duration: durationDays,
      active: true,
      id: this.generateId()
    };

    this.licenses.set(userId, license);
    return license;
  }

  generateId() {
    return `LIC-${Date.now()}-${Math.random().toString(3
6).substr(2, 8).toUpperCase()}`;
  }

  renewLicense(userId, extraDays = 30) {
    const license = this.licenses.get(userId);
    if (!license) return null;

    const newExpiry = new Date(license.expiresAt);
    newExpiry.setDate(newExpiry.getDate() + extraDays);

    license.expiresAt = newExpiry;
    license.active = true;
    license.renewedAt = new Date();

    return license;
  }

  isLicenseValid(userId) {
    const license = this.licenses.get(userId);
    if (!license) return false;
    if (!license.active) return false;

    const now = new Date();
    return now < license.expiresAt;
  }

  getExpiringLicenses(daysThreshold = 7) {
```

```
    const now = new Date();
    const threshold = new Date(now);
    threshold.setDate(threshold.getDate() + daysThreshol
d);

    const expiring = [];
    for (const [userId, license] of this.licenses) {
      if (license.active && license.expiresAt >= now &&
license.expiresAt <= threshold) {
        expiring.push({ userId, ...license });
      }
    }
    return expiring;
  }

  getLicenseStatus(userId) {
    const license = this.licenses.get(userId);
    if (!license) return { status: 'not_found' };

    const now = new Date();
    const isActive = now < license.expiresAt && license.
active;
    const daysRemaining = Math.floor((license.expiresAt.
getTime() - now.getTime()) / (1000 * 60 * 60 * 24));

    return {
      status: isActive ? 'active' : 'expired',
      active: isActive,
      daysRemaining: Math.max(0, daysRemaining),
      expiresAt: license.expiresAt,
      issuedAt: license.issuedAt
    };
  }
}

const licenseManager = new LicenseManager();
licenseManager.createLicense('user123', 30);
console.log(licenseManager.isLicenseValid('user123'));
```

```
// true
console.log(licenseManager.getLicenseStatus('user123'));
```

5. **Sistema de métricas y estadísticas temporales**:

```
class TimeMetrics {
  constructor() {
    this.data = [];
    this.sessionStart = new Date();
  }

  recordEvent(name, value = 1) {
    this.data.push({
      name,
      value,
      timestamp: new Date()
    });
  }

  getMetricsByDate(date) {
    const target = new Date(date);
    target.setHours(0, 0, 0, 0);
    const nextDay = new Date(target);
    nextDay.setDate(nextDay.getDate() + 1);

    return this.data.filter(d =>
      d.timestamp >= target && d.timestamp < nextDay
    );
  }

  getMetricsByHour(date) {
    const target = new Date(date);
    target.setMinutes(0, 0, 0);
    const nextHour = new Date(target);
    nextHour.setHours(nextHour.getHours() + 1);

    return this.data.filter(d =>
```

```
      d.timestamp >= target && d.timestamp < nextHour
    );
  }

  getDailyStats(date) {
    const dayMetrics = this.getMetricsByDate(date);
    const stats = {};

    dayMetrics.forEach(d => {
      stats[d.name] = (stats[d.name] || 0) + d.value;
    });

    return {
      date: date.toISOString().split('T')[0],
      totalEvents: dayMetrics.length,
      stats,
      hourlyDistribution: this.getHourlyDistribution(dat
e)
    };
  }

  getHourlyDistribution(date) {
    const distribution = {};
    const start = new Date(date);
    start.setHours(0, 0, 0, 0);

    for (let hour = 0; hour < 24; hour++) {
      const hourStart = new Date(start);
      hourStart.setHours(hour, 0, 0, 0);
      const hourEnd = new Date(hourStart);
      hourEnd.setHours(hour + 1, 0, 0, 0);

      distribution[hour] = this.data.filter(d =>
        d.timestamp >= hourStart && d.timestamp < hourEn
d
      ).length;
    }
```

```
    return distribution;
  }

  getTimeRangeStats(startDate, endDate) {
    const start = new Date(startDate);
    const end = new Date(endDate);

    const filtered = this.data.filter(d =>
      d.timestamp >= start && d.timestamp <= end
    );

    return {
      startDate: start,
      endDate: end,
      totalEvents: filtered.length,
      dateRange: `${start.toISOString().split('T')[0]}
-${end.toISOString().split('T')[0]}`,
      stats: filtered.reduce((acc, d) => {
        acc[d.name] = (acc[d.name] || 0) + d.value;
        return acc;
      }, {})
    };
  }
}

const metrics = new TimeMetrics();
metrics.recordEvent('page_view', 1);
metrics.recordEvent('click', 3);
metrics.recordEvent('page_view', 2);
console.log(metrics.getDailyStats(new Date()));
```

🧩 Relación con otros conceptos

- `Date.now()`: Método estático para obtener timestamp actual.
- `Date.parse()`: Convierte string de fecha a timestamp.
- `Date.UTC()`: Crea timestamp para fecha UTC específica.
- `Date.prototype`: Métodos de instancia (getFullYear, setHours, etc.).

- `Date.prototype.getTime()`: Obtiene el timestamp de la fecha.
- `Date.prototype.toISOString()`: Formato ISO 8601.
- `Date.prototype.toLocaleString()`: Formato localizado.
- `performance.now()`: Para mediciones de alta precisión.

🧠 Analogía

Imagina que `new Date()` es como un **relojero** que puede crear un reloj para cualquier momento:

- `new Date()`: Te da un reloj que marca la hora actual.
- `new Date(timestamp)`: Te da un reloj configurado a una hora exacta en milisegundos.
- `new Date('2024-12-25')`: Te da un reloj configurado para Navidad.
- `new Date(2024, 11, 25)`: Te da un reloj configurado con piezas separadas (año, mes, día).

📚 Fuente

- MDN Web Docs – Date
- ECMAScript Specification – Date
- MDN – Date constructor
- MDN – Date.parse()